LSTM-Based Volatility Prediction for Global X Uranium ETF

1.0 Introduction

Volatility forecasting is a fundamental problem in quantitative finance. The ability to predict periods of elevated market turbulence allows practitioners to adjust position sizing, hedge exposure, and manage risk. In this project, a Long Short-Term Memory (LSTM) network is used to predict the 21-day forward realized volatility of URA, the Global X Uranium ETF, using daily price and volume data. The model is benchmarked against GARCH, the standard econometric model for volatility estimation.

2.0 Background

This task is framed as a non-linear regression problem, where a rolling window of 60 days of engineered features is used as input to predict the standard deviation of log returns over the subsequent 21 trading days, which serves as a proxy for forward realized volatility. However, volatility forecasting remains challenging, as a substantial proportion of market movements is driven by external factors such as investor sentiment, macroeconomic announcements, and news events, which are not reflected in historical price data [3]. Therefore, both economic and machine learning models are subject to limitations in their predictive capacity, particularly in anticipating sudden volatility spikes. Furthermore, the uranium sector provides a particularly compelling case study for these limitations. As shown in Figure 1 below, the index experienced a long period of decline from 2011 to 2020 following the Fukushima disaster, with low and relatively stable volatility. From 2021 onwards, renewed interest in nuclear energy led to a strong market recovery, accompanied by much higher and more variable volatility. This regime shift between the training period (before 2021) and the test period (2023–2026) makes forecasting challenging.

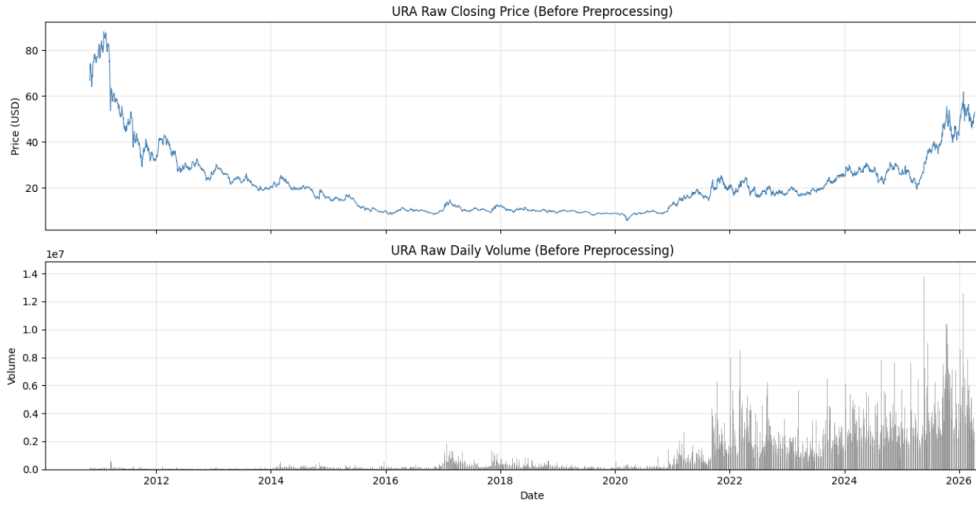

Fig 1. Raw closing price and daily trading volume for URA

3.0 Dataset

The dataset consists of daily Open, High, Low, Close, and Volume (OHLCV) data for URA, downloaded from Yahoo Finance via the yfinance Python library [1]. The data spans from URA's inception in November 2010 to April 2026, totaling approximately 3,860 trading days after feature engineering. URA is the Global X Uranium ETF, which tracks a basket of companies involved in uranium mining and nuclear energy production. The target variable is the 21-day forward realized volatility, defined as the standard deviation of log returns over the next 21 trading days.

3.1 Dataset Preprocessing

Raw OHLCV data cannot be used directly as inputs because prices are non-stationary and feature scales differ. To address this, 8 features were engineered from the raw data. Log returns are used to make prices more stable over time. The average of log returns over 5-day and 21-day periods captures short and medium-term trends. The standard deviation over the same periods measures recent volatility. Changes in volume reflect shifts in market activity. The High–Low range measures daily price variation, while the Open–Close range captures the direction and size of daily price movement. Furthermore, all features and the target were normalized using StandardScaler. In Figure 2, the effect of these transformations is evident. The top graph shows log returns fluctuating around zero with no visible trend, indicating that the non-stationarity present in price levels has been effectively removed. The middle panel presents the rolling volatility features (5 and 21-day), which clearly display periods of low and high volatility. The bottom panel shows the target variable, the 21-day forward volatility, which evolves more smoothly but still reflects distinct spikes during turbulent periods.

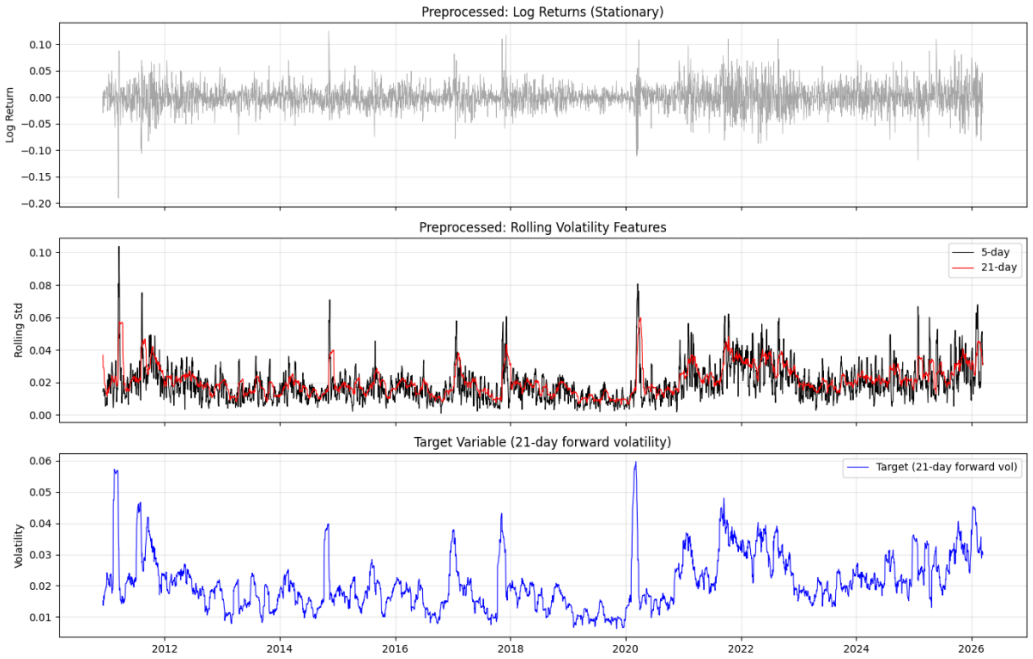

Fig 2. Preprocessed features — log returns, rolling volatility, and target vs GARCH baseline

3.2 Dataset Split

After preprocessing, the data was split chronologically into training (70%), validation (15%), and test (15%) sets. Chronological splitting was used to avoid look-ahead bias, as random shuffling would allow the model to learn from future data when predicting past values. Overlapping 60-day sliding windows were then constructed, where each window of 60 consecutive days of features maps to a single target value at the window's end.

4.0 Model

LSTM networks are a class of recurrent neural networks (RNNs) [4], where each prediction depends on a window of past observations processed in temporal order. Unlike standard feedforward networks, which treat inputs as independent, LSTMs maintain an internal memory state that is updated at each timestep, allowing them to learn which parts of the sequence are important for prediction. This makes them suitable for time-series tasks such as volatility forecasting, where recent market behavior influences outcomes. LSTMs process data using three gates — forget,

input, and output — to control what information is removed, stored, and passed forward. This structure allows the model to capture longer-term patterns in the data while avoiding issues such as the vanishing gradient problem. As illustrated in Figure 3, the model takes as input a sequence of 60 timesteps, each with 8 features. This input is passed through two LSTM layers with 64 and 32 units respectively, each followed by a dropout layer with a rate of 0.2 to reduce overfitting. The first LSTM layer returns sequences, allowing the second layer to further process the temporal information and learn more abstract patterns. The output is then fed into a dense layer with 16 neurons and ReLU activation, before passing to a final dense layer with a single neuron and linear activation, appropriate for a regression task where the predicted volatility is a continuous value.

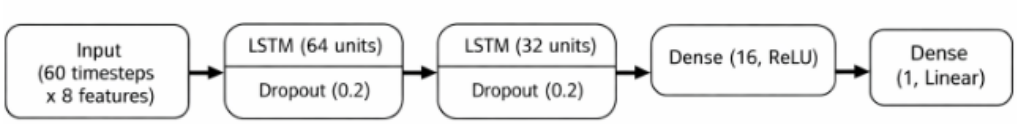

Fig 3. Model architecture diagram

4.1 Loss Function

Standard mean squared error (MSE) was used as the default loss function, but it led to poor behavior, with the model collapsing its predictions toward a near-constant value close to the mean volatility.

To address this, a custom loss was used that augments MSE with a variance-matching penalty:

$$L = MSE(y, \hat{y}) + 0.3 \cdot (1 - Var(\hat{y})/Var(y))^2$$

The penalty is minimized when the variance of the predictions matches that of the true values and increases as they diverge. This encourages the model to produce predictions with a similar spread to the target, preventing the flat-line issue while preserving MSE's focus on accuracy.

5.0 Training

The model was trained for up to 150 epochs using a batch size of 64. Optimization was performed using the Adam optimizer with an initial learning rate of 0.0001. To further improve convergence, ReduceLROnPlateau was implemented: if validation loss did not improve for 5 epochs, the learning rate was reduced by half, allowing the model to make smaller optimization steps as training progressed. Early stopping was applied based on validation loss with patience of 20 epochs to prevent unnecessary training.

Fig 4. Training and validation loss curves

The training plot shown in Figure 4 shows the learning behavior of the model over time. Both training and validation loss decreased sharply in the initial epoch, indicating the model quickly learns useful patterns from the data. However, after approximately epoch 5, the validation loss begins to increase gradually while training loss continues to decline, suggesting overfitting. This indicates that the model starts to fit noise in the training data rather than generalizable patterns. Early stopping halts training at epoch 24 and keeps the weights that achieved the lowest validation loss.

6.0 Results

To provide a meaningful benchmark for the LSTM, a standard GARCH(1,1) model was fitted to the URA log return series. GARCH (Generalized Autoregressive Conditional Heteroskedasticity) has been the dominant model for volatility since its introduction by Bollerslev [2], as it captures volatility clustering by modelling current variance as a function of past shocks and past variance.

6.1 True Labels vs Predictions

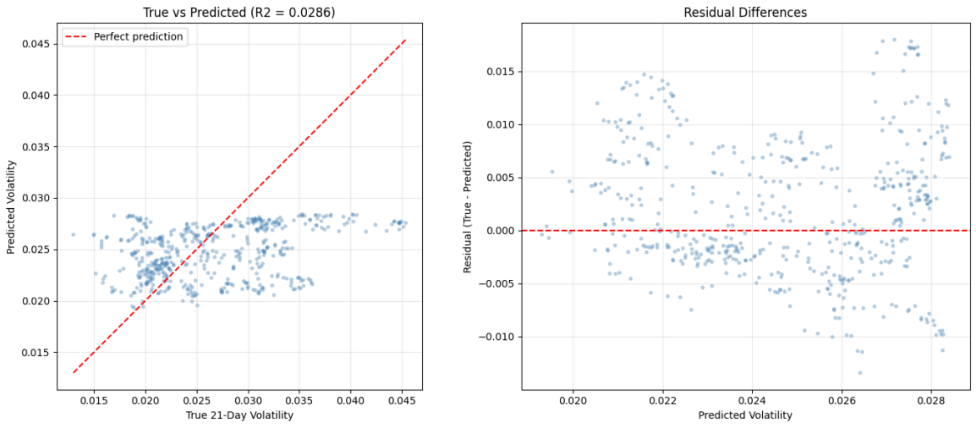

Fig 5. True vs predicted scatter plot (left) and residual differences (right)

Figure 5 above shows model performance on the test set through both predicted vs true values and residuals. The predictions follow a general upward trend with true volatility but are widely dispersed and remain within a narrow range, failing to capture extreme values. This indicates that the model is conservative and struggles during high-volatility periods. This is reinforced by the residual plot, where errors are centered around zero but increase as volatility rises, with more positive residuals at higher levels.

6.2 Test Set Performance Over Time

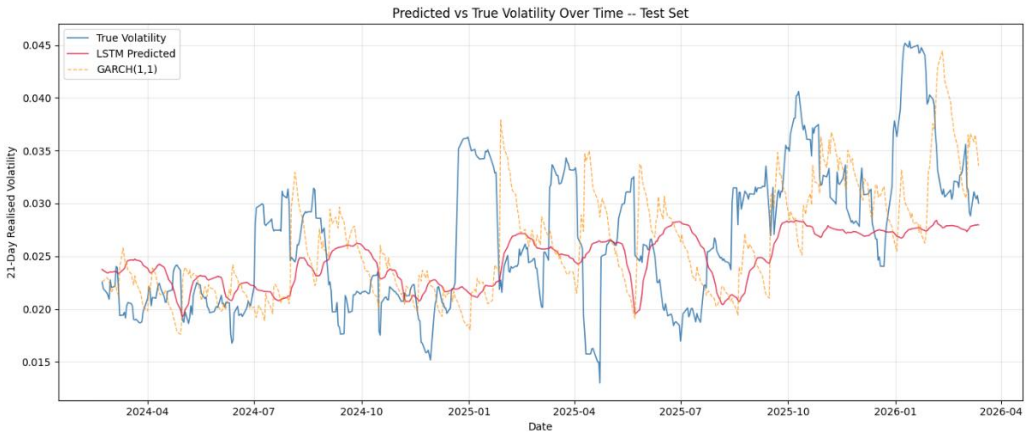

Fig 6. True vs predicted volatility over time with GARCH baseline

Figure 6 compares LSTM predictions with the GARCH(1,1) baseline, and true volatility over the test period. The LSTM captures the overall trend in volatility and reflects broad shifts between lower and higher volatility periods, but its predictions are relatively smooth and fail to fully capture sharp spikes. In contrast, GARCH is often noisy and does not consistently align with the longer-term movements in the 21-day forward volatility. Overall, LSTM provides a better representation of the underlying trend, while GARCH is more responsive but less stable.

6.3 Benchmark Comparison

Metric	LSTM	GARCH(1,1)
R^2	+0.04	−0.16
RMSE	~0.0065	~0.0071

The LSTM achieves a positive R-squared of +0.04 on the test set while GARCH produces a negative value of -0.16, indicating that the LSTM explains more variation in future volatility than a simple mean-prediction baseline, whereas GARCH performs worse than this baseline. This is further supported by the RMSE, where the LSTM (~0.0065) is lower than GARCH (~0.0071), showing that it produces more accurate predictions overall, although the improvement remains small given the difficulty of the task.

7.0 Conclusion

This project developed an LSTM network to forecast the 21-day forward realized volatility of URA, benchmarked against a GARCH model. The LSTM outperforms GARCH across both metrics. Despite this, the model is not capable of accurately predicting the target in a meaningful way, as predictions remain within a narrow range and fail to capture extreme volatility spikes. Additional experiments incorporating the VIX index and GARCH conditional volatility as supplementary inputs yielded no improvement, suggesting the LSTM had already extracted the available signal. Nevertheless, LSTM provides a meaningfully better forecast than the traditional GARCH approach.

References

1. Yahoo Finance, URA historical data. Available at: https://pypi.org/project/yfinance/ [Accessed April 2026].

2. Bollerslev, T. (1986). Generalized Autoregressive Conditional Heteroskedasticity. Journal of Econometrics, 31(3), pp.307–327.

3. Poon, S.H. and Granger, C.W.J. (2003). Forecasting Volatility in Financial Markets: A Review. Journal of Economic Literature, 41(2), pp.478–539.

4. Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), pp.1735–1780.